obsidian-mcp-router — quick reference

v0.116.7 · multi-vault Obsidian MCP server, shipped by its Claude Code plugin — one install = everything (54 slash commands · 57 MCP tools · 49 skills)
github.com/tboome33/obsidian-mcp-router

What it is for, in one minute

The problem. You keep your notes in **Obsidian** (a note editor working on Markdown files, grouped in folders called *vaults*). You work with **Claude Code** (Anthropic's coding assistant, in a terminal or in the desktop app). You would like Claude to read your notes, write some, search them — across *all* your vaults, not one — without handing it raw access to your disk and without configuring one server per vault.

The answer. `obsidian-mcp-router` is an **MCP server** — MCP (*Model Context Protocol*) is the standard through which an assistant like Claude calls outside "tools". This server exposes 56 tools (read a note, write, search, convert a PDF, build a graph of the links…) and **routes** each call to the right vault, hence the name. It talks to Obsidian through its HTTP API (the community plugin *Local REST API*), never by reading the files directly. It ships as a **Claude Code plugin** that installs and starts it: one install, everything is there.

What you actually do with it. You say "find my notes about X", "save this conversation into the wiki", "turn this PDF into a note", "where do I start in this vault?" — and Claude chains the right tools. The repository also provides a method for a **personal wiki kept by the assistant** (in the manner described by Andrej Karpathy: a catalogue, a journal, a "hot" cache of recent facts, typed pages) and the commands to maintain it.

The vocabulary you need to read the rest

`Vault`	A folder of Markdown notes opened in Obsidian. The router knows several, **local** (on this PC) or **remote** (an Obsidian running elsewhere, reachable over the network). Every tool takes a `vault` argument; without it, the current workspace's *default* vault is used.
`MCP server`	The Node.js program of this repository (`bin/obsidian-mcp-router.mjs`). Claude Code starts it and talks to it over its standard input/output ("stdio"). It carries the 56 tools.
`MCP tool`	A function Claude can call with JSON arguments (`get_file` , `write_file` , `search_smart` …). The full list is on the last page.
`Claude Code plugin` (**`obsidian-router`**)	The package installed into Claude Code from the repository's marketplace. It contains the MCP server *and* what teaches Claude to use it: 54 **slash commands**, 49 **skills**, 2 sub-agents, 11 **hooks**.
`Slash command`	A command you type in Claude Code (`/obsidian-router:meta-status`), or that Claude recognises in a French or

	English sentence ("diagnose the router"). Each is a procedure written for Claude that calls one or more tools.
`Skill`	The know-how page behind a command (`skills/<name>/SKILL.md`): the detailed procedure, the known traps, the refusals. Claude loads it when the task calls for it.
`Hook`	A small script Claude Code runs at a precise moment (session start, every message…). The plugin enables 3 by default: load the vault's hot cache, recall decisions already taken, say which vault this workspace is bound to.
`Local REST API`	The (community) Obsidian plugin that opens an HTTP API on each vault, with a port and a key. **Required** in every vault: everything the router reads or writes goes through it.
`Bridge (` **`mcp-router-bridge`** `)`	A second Obsidian plugin, written for this project, that adds to that API what it lacks: semantic search, template execution, opening a note with one click, atomic writes. **Optional**: without it, reading, writing and text search still work.
`Workspace`	The folder you opened Claude Code in (often a code repository). The router **binds** each workspace to a primary vault, optionally to secondaries: that is what decides where a write made without naming a vault lands.
`config.json`	`~/.claude/obsidian-mcp-router/config.json`: the list of your vaults (path, port, key), the remote vaults, the workspace bindings. It is *your* file, on your machine — never inside a repository.
`Wiki,` **`wiki-meta/`**	The note structure the `wiki-*` commands maintain in a vault: the pages under `wiki/`, and under `wiki-meta/` four scaffold files — `catalog.md` (the index), `journal.md` (the history), `hot.md` (the cache of recent facts, rewritten, never piled up), `overview.md` (the big picture).
`Conventions`	Sections of a vault's `CLAUDE.md` that tell Claude how to write *in this vault* (languages, provenance of sources, heading structure…). Installable one by one from a library of 14 snippets.
`Smart Connections, Templater`	Two third-party Obsidian plugins. The first computes *embeddings* (vectors that bring notes close by meaning): the router uses them for semantic search and twin-page detection. The second executes note templates.
`OKF`	*Open Knowledge Format* (v0.1, Google): an exchange format for note bases. The router exports a wiki as an OKF bundle, validates a bundle, and generates the OKF navigation (per-folder indexes) in the vault.

How the pieces fit together — the dependency chain

```
Obsidian  ←  Local REST API (community plugin)  ←  BRIDGE (mcp-router-bridge)
    ↑ HTTP per vault (port + apiKey of the Local REST API plugin)
MCP SERVER (obsidian-mcp-router) — Node process on the PC
    ↑ MCP over stdio, spawned by Claude Code
CLAUDE CODE PLUGIN (obsidian-router) — commands + skills + agents + hooks
```

Read the diagram bottom-up. You talk to Claude Code. The plugin taught it the commands and starts the MCP server. The server receives a tool call, picks the vault, and sends an HTTP request to that Obsidian's Local REST API plugin — or to the bridge, for the capabilities only a *running* Obsidian can provide.

Why HTTP rather than reading the files on disk? Because the router talks to a *live* Obsidian, not to inert Markdown. That gives it what only the running application has: Smart Connections' embeddings (search by meaning, `search_smart`), the real rendering of a Templater template (`execute_template`), opening a note in the window (click-to-open). A write goes through Obsidian's own change pipeline instead of racing a human editing the same file at that moment. And the same HTTP request reaches a vault on a NAS or behind a WireGuard tunnel exactly like a local one. The price: Obsidian must be running, and each vault has a port and an API key to keep track of — which is what `config.json` and `setup-vault` handle for you.

Bridge — runs *inside* Obsidian; it grafts its routes onto the Local REST API's HTTP server: `/search/smart` (semantic search), `/templates/execute` (Templater), `/open/*` and `/ping` (click-to-open and the self-test "which vault answers on this port"), `PUT /vault-cas/*` (≥ 0.7.0 — *atomic* conditional writes: "write only if the file has not changed since I read it"), `GET /smart-env/sources` (≥ 0.9.0 — serves Smart Connections' vector store, a hidden folder the REST API refuses to serve itself, which makes twin-page analysis possible on a remote vault). **Optional**: without it, basic CRUD works; `ifMatch` writes fall back to a verified but non-atomic path. It updates through BRAT from the GitHub releases.

MCP server — a Node ≥ 20.19.0 process on the PC, started by Claude Code (through the plugin — the normal case — or through a manual entry in `~/.claude.json`, for development). Requires the **Local REST API** plugin in **every** vault; its vault registry lives in `~/.claude/obsidian-mcp-router/config.json`.

Claude Code plugin — its commands and skills orchestrate the server's tools; two of its hooks read the vault straight from disk, because they run before any tool call (`hot-cache-load`, `decisions-recall`). It **ships and starts the server**: one install = everything, one update = everything.

What the router can do — the big families, and why they exist

- **One server for all your vaults.** Every tool takes a `vault` argument; `vault: "*"` runs a search on every vault at once ("fan-out"). No more one MCP server per vault.
- **Local and remote vaults treated alike.** A remote vault is a URL and an API key in `config.json` — Obsidian may run in a container on a server, on another machine on the network. Since v0.97.0, a

remote vault whose files are *also* visible from here (a mounted share) can declare it (`localPath`), which lets this machine install its plugins and read its hot cache.

- **Workspace → vault binding.** Which vault does a code repository open? Its own `.env` no longer decides (a cloned repository would carry someone else's decision): it is a binding confirmed once, on *your* machine, in `config.json` (`confirm_workspace_binding`, or `setup-vault --attach`). A project's `.env` can only **propose**; the session-start briefing says what this workspace is bound to and how to change it.
- **Lock mode.** Restricts a session to a single vault (safety, focus): any call to another vault is refused.
- **Wiki auto-enrichment.** At three natural moments of a conversation (a substantial answer, a decision taken, a change of topic), Claude offers to file it into the wiki. Four modes: `ClaudeAsk` (it asks, the default) · `Hybrid` · `FullAuto` (it saves without asking) · `off`.
- **Conversion toolbox.** PDF, DOCX, XLSX, PPTX, image (OCR), audio (transcription), YouTube video (captions), web page, git repository → Markdown, ready to be written into a note. The main engine, **MarkItDown**, is a Python extra installed once (`npm run install-markitdown`); without it the tools stay visible and return the install instruction. **Docling**, a second extra (`npm run install-docling`), does high-fidelity PDF (tables, layout), and `pdf_to_images` renders pages as images so the model can *see* them. A **relevance filter** (BM25, the classic search-engine formula) drops off-topic blocks before synthesis.
- **Knowledge graph.** The router builds a typed JSON graph of the links between pages, derives a reading tour ("where to start"), a page's neighbours, the shortest path between two pages; it exports the wiki as `llms.txt` or as an OKF bundle.
- **Wiki analysis.** `find_boundary_pages` spots the "frontier" pages — heavily linked but nearly empty, the ones worth writing; `find_twin_pages` spots pairs of near-identical pages by vector similarity, with a threshold computed on *your* vault — and works on a remote vault too (bridge ≥ 0.9.0).
- **Click-to-open.** Every write returns a link; one click opens the note in Obsidian (the bridge's `GET /open/<path>` route). `open_in_obsidian` does the same without a browser (ideal in Claude Desktop). For a vault whose disk is not visible, the link's port is declared in the config (`insecurePort`), and `GET /ping?v=<vault>` proves with one click *which* vault answers on a port.
- **Vault creation wizard.** `plan_vault` (a readable plan, with defaults and warnings) → adjustments → `provision_vault` (plugins, port, key, wiki, registry): in the simple case, a single interaction.
- **Multi-tenant mode** (optional): when one router instance serves several people (behind a proxy, on MCPHub), environment variables scope it — vault allowlist, read-only, per-user audit trail.
- **11 hooks.** 3 active by default for every plugin installer (`hot-cache-load`, `decisions-recall`, `workspace-briefing`, declared in `hooks/hooks.json`); the other 8 are opt-in (wired at the end of `setup-vault`, `--no-hooks` to skip, `node scripts/setup-vault.mjs --install-hooks` to add them later): automatic session journal, hot-cache size guard, wiki autocommit, link checker, doc-drift detector, update check…

Install, then attach a vault — the normal path

1. **Install the `obsidian-router` plugin in Claude Code.** That is all: the MCP server ships inside it and starts automatically. (Cloning the repository + `npm link`, via `/obsidian-router:meta-setup`, is

the path for the router's developers.)

2. **Have Node.js ≥ 20.19.0.** Python 3.10+ is only required for the conversion extras (`npm run install-markitdown` , `npm run install-docling`).
3. **Know whether those extras are there *before* a call fails.** Every `list_vaults` response carries `conversionToolbox` (`available` , `via` , `verified` , `hint`) and `meta-status` shows it on one line. Eight tools stop without MarkItDown; `youtube_to_markdown` falls back to captions through yt-dlp (if installed); `git_repo_to_markdown` (repomix) and `pdf_to_markdown_docling` are not affected. To stop being offered it: `OBSIDIAN_ROUTER_SKIP_MARKITDOWN=1` .
4. **The simplest path:** `/obsidian-router:meta-attach-vault` **.** An interactive wizard that attaches a vault to the current workspace (the common case), creates a standalone vault, or registers a remote vault. It installs the required Obsidian plugins, sets up the wiki, writes the `.env` , offers the conventions to install, and ends with a **verified final state**: plugins, wiki, conventions — and the ordered list of what is missing, if anything is.
5. **Command-line alternative:**

   ```
   npm run setup-vault -- "C:\VAULTS\MyVault"
   ```

   installs and configures the **Local REST API** + bridge plugins, writes the `.env` , allocates a unique port, adds the vault to the registry, wires the hooks (`--no-hooks` to skip, `--install-hooks` to do it later). `--attach <name>` binds the current workspace to a vault already known; for a remote vault whose files are visible from here, `--attach <name> --local-path <dir>` checks that the directory really is that vault before recording it.
6. **The bridge** (`obsidian-mcp-router-bridge`) is required for semantic search (with Smart Connections), template execution (with Templater) and click-to-open links. Without it: reading, writing and text search work; those three capabilities do not. `--install-plugins <vault> --dry-run` shows what is missing, then installs it from the official GitHub releases with the seal the preview printed; `--plugin-health <vault>` says what Obsidian actually loaded.
7. **Check:** `/obsidian-router:meta-status` , or simply say "*diagnose the router*".

Configuration — the workspace `.env` (the ONLY keys the router takes from a repository's file; everything else there is ignored and named on its error output)

Why this rule. A `.env` travels with a cloned repository: what it holds was written by someone else, on another machine. So the router only takes *reading preferences* and *proposals* from it; anything that grants a right (writing without asking, locking, scoping an instance) comes from the MCP host or from your `config.json` .

`VAULT_PATH`	**What it is for:** saying which vault this workspace uses, portably. Path of the current vault; if it matches a registered vault, that vault becomes the default. Written by `setup-vault.mjs` .
`OBSIDIAN_ROUTER_DEFAULT_VAULT`	**What it is for:** choosing a process's default vault. **From the host only** (MCP server declaration, launcher, shell), where it wins over `VAULT_PATH` and `config.defaultVault` . The same variable

	in a project's `.env` is a **proposal**: reported by `list_vaults` in `bindingHint`, never applied. What decides is the workspace's confirmed binding in your `config.json` (`workspaceBindings`) — `confirm_workspace_binding` or `setup-vault --attach` records it.
`OBSIDIAN_ROUTER_LOCKED`	**What it is for:** working in one vault, with no risk of writing elsewhere. Any call to another vault fails. Set via `/obsidian-router:lock <vault> --persist`. Same origin rule: authority from the host, proposal from a project file. `--persist` records `locked: true` on the binding (read again at restart); the `.env` line it also writes is a portable hint for another machine.
`OBSIDIAN_ROUTER_AUTO_ENRICH`	**What it is for:** setting how much Claude files into the wiki on its own. `ClaudeAsk` (default) | `Hybrid` | `FullAuto` | `off`. Set via `/obsidian-router:auto-mode <Mode> --persist`. **One value is refused from a workspace file:** anything that reads as `FullAuto`, because it is a standing permission to write without asking again, and a cloned `.env` has no business granting it. `FullAuto` comes from the server declaration in the host, or from a `set_auto_enrich_mode` call in the session; a refusal is named on the error output and carried by `list_vaults` (`autoEnrichModeRefused`).
`MD_ALLOWED_PATHS` · `MD_SHARE_DIR`	**What it is for:** limiting what the conversion tools may read and where they may write on the local disk (a sandbox). Absent = no sandbox. The two names are ONE setting; a workspace file can only *tighten* it.
`OBSIDIAN_ROUTER_NO_*`	**What it is for:** switching off a convenience (never a guard). The 14 opt-outs enumerated in `src/helpers/workspace-dotenv.mjs`, accepted by name: NO_WATCH (config hot-reload), NO_HOT_CACHE_LOAD, NO_DECISIONS_RECALL, NO_WIKI_QUERY_FIRST, NO_WIKI_AUTOCOMMIT, NO_SESSION_JOURNAL, NO_HOT_CACHE_GUARD, NO_LINT_VAULT_LINKS, NO_DOC_STARTUP_CHECK, NO_DOC_PROPAGATION_CHECK, NO_UPDATE_CHECK, NO_AUTO_INSTALL_HOOKS, NO_AUTO_CONFORMANCE, NO_OKF_PROJECTIONS. Values: `1` / `true` / `yes` / `on`.

Configuration — host variables (never taken from a workspace `.env`: set them in the server declaration on the MCP host, in the launcher of a served instance, or in the shell)

`OBSIDIAN_ROUTER_ALLOWED_VAULTS`	**What it is for:** showing an instance only some of the vaults (shared deployment). Allowlist, comma-separated; the others are skipped.

`OBSIDIAN_ROUTER_READONLY`	**What it is for:** an instance that cannot change anything. Truthy value (`true` / `1` / `yes` / `on`): the **19 write tools** (write_file, append_to_file, patch_file, set_frontmatter, merge_frontmatter, move_file, delete_file, execute_template, download_page_assets, pptx_extract_assets, build_wiki_graph, provision_vault, register_remote_vault, unregister_vault, write_bundle, record_source, install_conventions, refresh_okf_projections, build_search_index) are removed from the tool list *and* refused when called.
`OBSIDIAN_ROUTER_USER_ID`	**What it is for:** knowing who wrote what when several people go through the same instance. Every successful write appends an attributed line to the touched vault's `wiki-meta/journal.md` . Also hides the local-only tools (`plan_vault` , `provision_vault`).
`VAULT_<NAME>`	**What it is for:** declaring a vault without a config file — handy on a server administered from a dashboard (MCPHub). A whole vault as JSON: `name` , `baseUrl` , `apiKey` , and for a remote vault `obsidianName` (its exact name inside Obsidian, for the open links).
`OBSIDIAN_ROUTER_CONFIG` · `MARKITDOWN_PATH` · `DOCLING_PATH` · `REPOMIX_PATH` · `YTDLP_PATH` · `HTTPS_PROXY` · `HF_HOME`…	**What it is for:** saying where the config and the external tools are. Read from the host's environment only (README § environment). A spawned tool receives a named list of variables, never the router's whole environment.

Important files — where things live

`~/.claude/obsidian-mcp-router/config.json`	The router's main config: vaults and port registry, default vault, disabled vaults, remote vaults, workspace bindings. Hot-reloaded when it changes.
`~/.claude.json`	Developer path only. The plugin declares the server (key `router`) in its root `.mcp.json` via `${CLAUDE_PLUGIN_ROOT}` . An extra manual `mcpServers.obsidian-router` entry runs **two processes** with duplicated tools (some dev machines do that on purpose); `claude --plugin-dir <checkout>` replaces the installed plugin for one session.
`~/.claude/settings.json`	The **8 opt-in hooks** (out of 11) are declared here; the 3 default ones come from `hooks/hooks.json` inside the plugin. The plugin itself is enabled **per workspace** via `<workspace>/.claude/settings.json` — 54 commands + 49 skills ≈ 10k tokens of context, to be loaded only where you use Obsidian.

`<workspace>/.env`	Read at router start — but only the workspace keys above are taken, the parent environment always wins, and any other key is ignored and named once on the error output.
`<vault>/CLAUDE.md`	The vault's instructions, loaded into context by Claude Code when the session is opened *inside* the vault: the auto-enrichment mode and the installed conventions. From a code workspace they are not loaded: the router's first write into the vault returns them (`vaultConventions`).
`<vault>/wiki-meta/{catalog,journal,hot,overview}.md`	The wiki scaffold: catalogue, journal (append-only), hot cache, overview — under `wiki-meta/` ; the pages themselves are under `wiki/` . Named `catalog` / `journal` rather than `index` / `log` because OKF reserves those two names; the old ones are still read. Created by `/obsidian-router:wiki` .
`<vault>/wiki-meta/Sessions/`	The detail of each session (written by the `session-auto-journal` hook); `journal.md` keeps one line of it.
`obsidian-mcp-router/docs/`	`auto-enrichment.md` (the 4 modes and the 4 filing channels), `features/` (13 prose sheets), `architecture.md` , `remote-vaults.md` (remote vaults), this page (FR + EN).

54 slash commands /obsidian-router:<name> · also recognised in natural language (FR + EN)

A slash command is a procedure written for Claude. You do not need to type it: the third column gives a sentence that triggers it. They fall into five groups by *what you want to do*.

🔧 17 MCP wrappers — one command per basic vault tool

When: you want one precise operation on a note — read, write, search, move, convert. Each wrapper calls one tool and shows you the result. Categories: `discover` · `read` · `write` · `manage` · `template` · `convert`

COMMAND	EFFECT	TRIGGER PHRASE (FR/EN)
`discover-list-vaults`	Lists the vaults (online or not, latency, lock state)	« liste mes vaults » / "list my vaults"
`discover-list-files`	Lists the files of a vault folder	« liste les fichiers de X » / "list files in Sessions"
`read-get`	Reads a file (Markdown + frontmatter)	« montre-moi X » / "show me X"
`read-search`	Plain text search (substring)	« trouve <texte> » / "grep for <text>"
`read-search-smart`	Search by meaning (Smart Connections)	« trouve mes notes sur X » / "find notes about X"
`read-frontmatter`	Reads the frontmatter (all of it, or one key)	« quel est le statut de X » / "metadata of X"
`write-create-or-replace`	Creates or replaces a file	« crée une note X » / "save this as X.md"
`write-append`	Appends at the end of a file	« ajoute à X » / "append to my journal"
`write-patch`	Surgical edit (a heading, a block, the frontmatter)	« édite la section X dans Y » / "edit X section in Y"
`write-frontmatter-set`	Sets one frontmatter key	« passe le statut de X à closed » / "tag this with X"
`write-frontmatter-merge`	Sets several keys at once	« sur X mets status=closed outcome=tp1 »
`write-bundle`	Several files in one go — all or nothing, with a journaled rollback	« écris ces 4 pages d'un bloc » / "write these pages atomically"

`manage-move`	Moves or renames a file	« renomme X en Y » / "move X to Y"
`manage-delete`	Deletes a file (two-step confirmation)	« supprime X » then « oui confirm=true »
`template-execute`	Runs a Templater template	« exécute le template daily » / "run X template"
`pdf-to-markdown`	Local PDF → Markdown (MarkItDown, plain text, fast)	« convertis ce PDF en markdown » / "convert this PDF"
`pdf-to-markdown-docling`	PDF → high-fidelity Markdown (Docling: tables, layout; extra to install)	« conversion haute fidélité » / "convert with docling"

🔒 4 router-state commands (lock, auto-enrichment, port base)

When: you want to change how the router behaves during this session — which vault it works in, how much it files on its own, which ports it will assign to the next vaults.

COMMAND	EFFECT	TRIGGER PHRASE (FR/EN)
`lock`	Restricts the session to one vault (for the session, or `--persist`)	« verrouille sur X » / "I only want to work on X"
`unlock`	Lifts the lock, restores multi-vault routing	« déverrouille les vaults » / "unlock vaults"
`auto-mode`	Sets the auto-enrichment mode (ClaudeAsk / Hybrid / FullAuto / off)	« passe en mode Hybrid » / "switch to Hybrid mode"
`force-new-port-start`	Draws a new port-allocation base for **future** vaults only — two-phase sealed plan, no existing port changed	« tire une nouvelle base de ports » / "draw a new port base"

🔗 2 workspace-binding commands (which vault this project writes into)

When: you open Claude Code in a project folder and want "the vault" to mean the right one without naming it on every call — and decide what this project may write into other vaults.

COMMAND	EFFECT	TRIGGER PHRASE (FR/EN)
`bind-workspace`	Wizard — binds this workspace to a PRIMARY vault, then optionally to secondaries with a write tier each; also names an unanswered proposal from the project's	« à quel vault ce projet est-il rattaché » / "bind this workspace to a vault"

	`.env`, and records a "no" as a refusal	
`configure-secondary-vaults`	The same wizard, entered at its SECONDARIES step: declares the secondaries and picks each tier — strict read-only / write on request / read-write	« ajoute un vault secondaire » / "make X read-only for this project"

🩺 7 conversational helpers (setup + diagnostics)

When: something needs installing, checking or repairing — a vault to attach, a missing plugin, an unreachable vault, conventions to propagate.

COMMAND	EFFECT	TRIGGER PHRASE (FR/EN)
`meta-setup`	Manual router install (clone + npm link) — developer path; a normal install gets the server through the plugin	« installe le router » / "setup obsidian-mcp-router"
`meta-attach-vault`	Wizard — attaches a vault to a workspace (the common case), creates a standalone vault, or registers a remote vault; ends with the verified final state	« attache un vault à ce workspace » / "connect my remote vault"
`meta-status`	Health check of every vault, with fix hints; also says which router build answers and whether the session's hooks ran	« diagnostique le router » / "are my vaults reachable"
`meta-sync-template`	Propagates plugins, snippets and docs from the reference vault to other vaults (interactive picker)	« synchronise le template » / "sync the template to all vaults"
`sync-from-github`	Updates one vault or the whole fleet from the skeleton published on GitHub (plugins, themes, snippets, docs) — no local development repo needed	« synchronise depuis GitHub » / "sync this vault from github"
`meta-audit-bridge-readiness`	Checks that click-to-open is ready (bridge, REST API, HTTP, live `/open` probe)	« audite le bridge » / "is click-to-open ready"
`conventions`	Installs / removes / lists / audits / propagates the conventions of a vault's `CLAUDE.md`; migrates the	« installe la convention X » / "install source-type on X"

old `bilingual` convention to `languages`

📚 24 knowledge-management commands (the wiki kept by the assistant)

When: you build and maintain a note base with Claude — pour sources into it, query it, check it, compact it, visualise it, export it.

COMMAND	EFFECT	TRIGGER PHRASE (FR/EN)
`wiki`	Sets up the wiki in a vault (catalogue, journal, hot cache, overview) and offers the conventions	« scaffold un wiki » / "set up a wiki"
`wiki-ingest`	Pours a source (URL, file, text) into the wiki → entity and concept pages, cross-linked	« ingère cette URL » / "absorb this article"
`wiki-query`	Answers from the wiki (three search tiers, no web)	« que dit mon wiki sur X » / "based on my notes …"
`wiki-lint`	Health check: orphan pages, dead links, stale catalogue	« lint le wiki » / "audit my wiki"
`wiki-fold`	Compacts the journal into periodic summaries under `wiki/folds/` (re-runnable without duplicates)	« compacte le journal » / "fold the log"
`hot-compact`	Brings an oversized `hot.md` back to its cache contract (verified backup first)	« compacte le hot » / "compact the hot cache"
`save`	Files the conversation as a typed note (session, decision, ADR…)	« sauvegarde ça » / "save this"
`decision-consolidate`	Reduces a settled decision page to its essentials and moves the full deliberation into a verified archive note	« consolide cette décision » / "consolidate this decision"
`autoresearch`	Autonomous loop: web search → synthesis → filing, until the questions run out	« fais une recherche web sur X » / "research X online"
`canvas`	Creates or edits Obsidian `.canvas` files (visual boards)	« crée un canvas pour X » / "create a canvas for X"
`defuddle`	Strips the noise from a web page (menus, ads) before ingesting it	« nettoie cette page » / "defuddle <url>"

`obsidian-bases`	Creates or edits Obsidian `.base` files (database-like views)	« crée une base pour X » / "create a base for X"
`wiki-graph`	Builds the link graph (typed JSON, readable by Obsidian's native viewer)	« construis le graphe » / "build the wiki graph"
`wiki-tour`	An ordered reading tour, derived from the link topology	« par où je commence » / "give me a tour of this vault"
`wiki-neighbors`	A page's neighbours — outgoing links, backlinks, or both	« voisins de X » / "what links to X"
`wiki-path`	The shortest link chain between two pages	« quel rapport entre X et Y » / "how is X connected to Y"
`wiki-export`	Exports the vault as `llms.txt` / `llms-full.txt` or as an OKF bundle	« exporte le wiki » / "export the wiki as llms.txt"
`okf-export`	Exports part of the wiki as a shareable OKF v0.1 bundle (conformance checked)	« publie en bundle OKF » / "export this folder as an OKF bundle"
`okf-projections`	Regenerates the OKF navigation in `wiki/` (root and per-folder indexes, newest-first log); refreshed automatically after writes; `--check` = drift report	« regénère les index du wiki » / "refresh the OKF projections"
`okf-check`	Validates any OKF bundle against the v0.1 conformance rules	« ce bundle est-il conforme ? » / "validate this OKF bundle"
`wiki-refresh-digests`	Regenerates each page's sidecar digests (concepts, claims, keywords)	« rafraîchis les digests » / "refresh the digests"
`build-search-index`	Builds or refreshes the local BM25 search index (no plugin; re-runnable)	« construis l'index de recherche » / "build the search index"
`wiki-boundary`	Ranks the "frontier" pages — heavily linked but nearly empty, the ones worth writing	« qu'est-ce que je devrais écrire ensuite » / "frontier pages"
`who-is-speaking`	In a shared family vault, identifies who is speaking and files their notes in their folder	« c'est Karine » / "who is speaking"

The 57 MCP tools — what Claude actually calls, and what each is for

You do not call these tools yourself: the commands and skills do. This table is for understanding what happens, and for reading the responses. Every tool accepts `vault`; the writes accept `ifMatch` (see "Writes").

`list_vaults` · `list_files`	**Discovery — knowing what exists.** `list_vaults` is the call to make first: the catalogue of vaults, online or not, with latency, the router build that answers (`routerBuild`), whether the session's hooks ran (`sessionHooks`), the state of the conversion extras (`conversionToolbox`) and the workspace binding. `list_files` lists a folder.
`get_file` · `search` · `search_smart` · `get_frontmatter`	**Reads.** The whole file (returns `contentSha256`, the fingerprint to replay as `ifMatch` to write without clobbering); text search; search by meaning (Smart Connections embeddings, cosine scores); typed frontmatter. `vault: "*"` = across all vaults. `search_smart` says two things about its own answer: `freshness` flags a hit whose page changed since it was indexed (`changed` / `touched`) or vanished (`page-missing`) — open the page before citing it; `folderExclusion` says what was left out (by default `wiki-meta/Sessions`, the chronological journals; `excludeFolders: []` brings them back).
`write_file` · `append_to_file` · `patch_file` · `delete_file` · `set_frontmatter` · `merge_frontmatter` · `move_file`	**Writes and file management.** Create or replace; append; edit a heading, a block or the frontmatter; delete (guarded by `confirm: true`); one or several frontmatter keys; move or rename. `ifMatch`: replay the `contentSha256` read by `get_file` and the write is refused (409) if the file changed in between — that is what protects a note someone else is editing; atomic with bridge ≥ 0.7.0, verified fallback otherwise. A vault shared between several workspaces requires it (`writesRequireIfMatch`).
`execute_template`	**Templater.** Renders a template and writes the result if asked; values are passed via `tp.mcpTools.prompt("key")`.
`lock_vault` · `unlock_vaults` · `set_auto_enrich_mode` · `confirm_workspace_binding` · `set_secondary_vault_mode`	**Router state and workspace binding.** Single-vault isolation; auto-enrichment mode; confirm, refuse or clear this workspace's binding (`{ vault }`, `{ vault, also: [...] }` for several, `{ refuse: true }`, `{`

	`clear: true }`) — recorded in *your* `config.json`, never in the project. `set_secondary_vault_mode` sets a secondary's **write tier**: `locked` (server-side refusal), `soft` (the default: refused until a `confirmSecondaryWrite` confirmation is given), `writable`. On a shared instance (`READONLY` / `ALLOWED_VAULTS` / `USER_ID`), refused: a persisted answer would speak for everyone.
`plan_vault` · `provision_vault` · `register_remote_vault` · `unregister_vault`	**Creating or declaring a vault.** A read-only plan (defaults, questions, warnings) → creation in one call (plugins, port, key, wiki, registry) — the vault is judged on its real path, pinned for the whole run, and refused if its tree holds a link. `register_remote_vault` registers from the conversation a remote vault already in service (name, `baseUrl`, `apiKey`, and if its files are visible from here `localPath`) — in `config.json`, never in a project's `.env`; hidden on a shared instance.
`pdf/docx/xlsx/pptx/image/audio_to_markdown`	**Local conversion** (MarkItDown, an extra to install: `npm run install-markitdown`; the tools stay listed and return the install instruction if it is missing). Office files, PDF, image OCR, audio transcription → Markdown, to chain with `write_file`.
`pdf_to_markdown_docling` · `pdf_to_images`	**High-fidelity PDF** (Docling, extra): layout and tables preserved. `pdf_to_images` renders pages as PNG so the model can *see* them (page count and bytes capped).
`pptx_extract_assets`	**A deck's pictures, on disk**, without Python (a PPTX is a ZIP): each embedded image written once, with the slides that show it — what `pptx_to_markdown` cannot render. A write tool: an `outdir` inside a vault is subject to that vault's tier; outside any vault, only the temp directory is allowed.
`youtube/bing_search/webpage_to_markdown` · `git_repo_to_markdown`	**Remote conversion.** URL → Markdown (with a guard against internal addresses); YouTube captions; a whole git repository → one document (repomix, optional Tree-sitter compression). `webpage_to_markdown` accepts `relevanceQuery` (keep only what answers a question) and `citations: true` (links become numbered footnotes).
`filter_relevant_blocks`	**Relevance filter.** A second, deterministic BM25 pass over Markdown already obtained: drops off-topic blocks, keeps frontmatter and headings. No network access, no language model.

`extract_page_metadata` · `propose_linked_sources` · `download_page_assets`	**Web-ingestion support.** A page's metadata (JSON-LD, OpenGraph); the links worth following, ranked; downloading the images into the vault.
`get_wiki_context_pack` · `build_wiki_graph` · `build_wiki_tour` · `get_page_neighbors` · `wiki_path`	**Context and graph.** A structured context pack for an outside agent; the typed graph; the reading tour; the neighbours; the shortest path. Every item of the pack says where it came from: `index` (the catalogue) and `graph` (a link) are **navigation, which is authoritative**; `semantic` is a complement, never the sole support of a fact. A pack with no navigation anchor says so: `answer-relies-on-semantic-only`.
`find_boundary_pages` · `find_twin_pages`	**Wiki analysis.** The "frontier" pages worth writing; the near-twin pairs (vector similarity, threshold computed on the vault), each with the hints to settle it. When they cannot look, both answer `available: false` with the reason — never a false "0 found". `find_twin_pages` also works on a remote vault (bridge ≥ 0.9.0).
`write_bundle` · `refresh_okf_projections` · `build_search_index`	**Wiki maintenance.** Several files in one go, all or nothing, with rollback; regeneration of the OKF navigation (`index.md` per folder, newest-first `log.md` — generated files, never to be edited by hand); local BM25 search index (every vault, no plugin required).
`record_source` · `audit_sources`	**Source ledger.** Where ingested content came from (URL, hash, date); audit of a vault's sources (drift, gaps).
`audit_vault_conventions`	**Conventions audit.** Which conventions a vault is really under: the file the router resolves (or the two it cannot choose between), the recommended ones missing, template copies, inherited backups — each finding with a repair to approve, never applied on its own.
`install_conventions`	**Conventions installer.** Installs library conventions by id into the vault's conventions file in ONE guarded write (texts read from the package, never re-typed by the model), then reads the file back and verifies; `languages` takes the vault's own value, a retired convention is reported and never written, `dryRun` returns the state the picker needs.

`build_open_link` · `open_in_obsidian` · `get_view_link`	**Navigation.** A ready-to-paste click-to-open link (path checked against the vault; on a vault without a disk the link is built anyway, marked `pathVerified: false`); server-side opening (the Obsidian window comes to the front, optional heading); an ephemeral view link to a remote Obsidian, for container deployments.

Tips · Type `/obsidian-router:` in Claude Code → autocomplete · Every command accepts a `vault` argument (otherwise, the workspace's vault) · `vault: "*"` on `search` / `search_smart` = every vault · Trigger phrases are indicative, Claude recognises variants · Write results carry a ready-made `clickToOpenUrl` — one click opens the note in Obsidian · **Tool names by installation**: same tools, three prefixes — plugin: `mcp__plugin_obsidian-router_router__<tool>` · manual registration: `mcp__obsidian-router__<tool>` · MCPHub: `mcp__<id>__obsidian-router—<vault>—<tool>`; the first two coexist (duplicated tools) if the manual entry is kept · Full docs: `README.md`, `docs/features/` (13 sheets), `docs/auto-enrichment.md`, `docs/remote-vaults.md` in the repository.